

QA Test Case Register

All 92 test cases executed against the Risk Assessment Workbench — preconditions, steps, expected and actual results — followed by the observations that came out of running them.

AUTHOR	EXECUTED	BUILD	ENVIRONMENT	
Shanthi Subramanian · BA / QA	12 September 2026	qa/ba-qa-workstream	Local · .NET 10.0.401	

92	80	9	12	6
TEST CASES	EXECUTED	KNOWN- DEFECT	BLOCKED	DEFECTS RAISED

How to read this

Conventions, ID scheme, and what each status actually means.

Status vocabulary

STATUS	MEANING
PASS	Executed; actual matched expected.
KNOWN DEFECT	Executed; actual did not match expected, and that mismatch is a raised defect. Marked <code>test.fail()</code> in code so the suite stays green while the gap is open — and reports <i>"expected to fail, but passed"</i> the moment it is fixed. These are failures, deliberately recorded.
BLOCKED	Could not execute. Currently only the AI eval cases, which need a configured model. Reported as <code>Skip</code> , never as <code>Pass</code> — "we did not check" is not the same claim as "it worked".

ID scheme

PREFIX	SUITE	CASES	AUTHORED BY
UT-AS-nn	Unit — AssessmentService	12	QA (new)
UT-xx-nn	Unit — Scoring, Committee, ChangeRequests, DataIngestion, AI clients, DocumentProcessing	50	Dev (inherited, executed & verified by QA)
API-SM-nn	Playwright — API smoke	3	QA (new)
API-AU-nn	Playwright — authorization	9	QA (new)
UI-JN-nn	Playwright — role journeys	3	QA (new)
UI-GV-nn	Playwright — governance defects	3	QA (new)
EV-xx-nn	AI evaluation datasets	12	Dev (inherited, executed by QA)

ON THE INHERITED CASES

Fifty unit tests and twelve eval cases existed before this QA pass. They are listed in full because QA *executed and verified* all of them — a register that only showed new work would misrepresent what has actually been checked. They are marked as inherited so authorship stays honest.

Environment under test

COMPONENT	VERSION / ENDPOINT
.NET SDK	10.0.401 (runtime 10.0.12)
PostgreSQL	postgres:16 · localhost:5433
Azurite (blob)	localhost:10000
Workbench API	localhost:5210
Mock Systems	localhost:5220
webapp (Vite 7.3.6)	localhost:3000
Playwright	1.63.0 · Chromium 153
Auth	Auth0 unconfigured — <code>DevBypassAuthHandler</code> active
AI provider	Not configured at execution time — eval cases blocked

Execution report

Results as run on 12 September 2026.

SUITE	CASES	PASS	KNOWN DEFECT	BLOCKED	RUNTIME
Unit (xUnit + Moq)	62	62		0	0.64 s
Playwright — api	12	6		6	3.0 s
Playwright — ui	6	3		3	41 s
AI evaluation	12	2		10	<1 s
Total	92	73		9	10

Stability

The Playwright suite was run three consecutive times end to end: **18 passed on every run**, no flakes. The unit suite was run four times across the session with identical results. One test was written and then deliberately removed for being inherently flaky — see OBS-3.

Suite integrity — the mutation check

A green suite is a claim, not evidence. The finalization gate was deliberately broken (`if (!readiness.IsReady)` → `if (false)`) and the unit suite re-run:

Failed: 5, Passed: 57, Total: 62

Exactly the five `FinalizeAsync_*` cases failed — UT-AS-01 through UT-AS-05 — while the seven readiness and delegation cases correctly stayed green, because they call `CheckReadinessAsync` directly and never traverse the gate. Production code was then restored byte-for-byte and the suite returned to 62/62.

Unit — AssessmentService

12 cases · `tests/.../Assessment/AssessmentServiceTests.cs` · authored by QA. All dependencies mocked; no database, no HTTP.

UT-AS-01 Finalization is blocked while a narrative section is still AI-drafted

PASS

COVERS

US-5.1 AC2 — an AI draft cannot route to committee

PRE

One active mapped category ("Products & Services") with a narrative section at status `AiDrafted`, and one policy reliance decision recorded

STEPS

1. Call `FinalizeAsync(assessmentId, analystId)`

EXPECTED

`InvalidOperationException` whose message contains "Narrative not yet reviewed" *and* names the category

ACTUAL

As expected — the category is named, not just refused

UT-AS-02 Finalization is blocked when a mapped category has no narrative at all

PASS

COVERS

US-6.3 AC1

PRE

One active mapped category; **no** narrative sections; one reliance decision

STEPS

1. Call `FinalizeAsync`

EXPECTED

Throws; the category is listed as outstanding

ACTUAL

As expected — absence is treated the same as unreviewed, not as "nothing outstanding"

UT-AS-03 Finalization is blocked when a category has no policy-reliance decision

PASS

COVERS

US-3.2 AC2

PRE

One active category with an `AnalystReviewed` narrative; **no** reliance decisions

STEPS

1. Call `FinalizeAsync`

EXPECTED

Throws with "No policy reviewed" naming the category

ACTUAL

As expected

UT-AS-04 A blocked finalization never reaches the repository

PASS

COVERS

The governance assertion itself

PRE

One category, narrative `AiDrafted`, no reliance — i.e. both gates failing

STEPS

1. Call `FinalizeAsync` and catch
2. Verify `IAssessmentRepo.FinalizeAsync` invocations

EXPECTED

`Times.Never`

ACTUAL

As expected. This is the case that matters most — throwing *after* persisting would still leave an unreviewed assessment in `PendingCommittee`

UT-AS-05 Every outstanding reason is listed, not just the first

PASS

COVERS

US-6.3 AC1 — "lists what's outstanding"

PRE

Two active categories; only the first has a reviewed narrative and a reliance decision

STEPS

1. Call `FinalizeAsync`
2. Inspect the exception message

EXPECTED

Message contains both "Narrative not yet reviewed" *and* "No policy reviewed", and names the second category

ACTUAL

As expected — an analyst fixing one complaint will not discover a second on resubmission

UT-AS-06 The finalizing analyst's identity is recorded

PASS

COVERS

US-6.3 AC3 — "cannot be anonymous or attributed to 'system'"

PRE

One category, narrative `AnalystReviewed`, one reliance decision — all gates satisfied

STEPS

1. Call `FinalizeAsync(assessmentId, analystId)`

EXPECTED

Repo called exactly once with *that* analyst id

ACTUAL

As expected

UT-AS-07/08 Both AnalystReviewed and AnalystEdited satisfy the narrative gate

PASS ×2

COVERS

US-6.3 AC2 — "accepted as-is or edited"

PRE

One category; narrative at the parameterised status; one reliance decision

STEPS

1. `[Theory]` over `AnalystReviewed` and `AnalystEdited`
2. Call `CheckReadinessAsync`

EXPECTED

`IsReady == true`; no outstanding sections

ACTUAL

As expected for both — only `AiDrafted` blocks

UT-AS-09 A category the analyst deactivated is ignored

PASS

COVERS

Epic 2 — category override

PRE

Two categories; the second has `IsActive = false` and no narrative or reliance

STEPS

1. Call `CheckReadinessAsync`

EXPECTED

Ready; the deactivated category appears in neither outstanding list

ACTUAL

As expected — a removed category cannot hold finalization hostage forever

UT-AS-10 An assessment-wide policy reliance satisfies every mapped category

PASS

COVERS

The `RiskCategoryId == null` branch

PRE

Two categories, both with reviewed narratives; a single reliance row with `RiskCategoryId = null`

STEPS

1. Call `CheckReadinessAsync`

EXPECTED

Ready; no category missing policy reliance

ACTUAL

As expected

UT-AS-11 An assessment with no mapped categories reports "ready"

CHARACTERIZATION · DEF-004

COVERS

Documents a defect, not a requirement

PRE

No categories, no sections, no reliance

STEPS

1. Call `CheckReadinessAsync`

EXPECTED

Per the product's own rule: should not be ready

ACTUAL

`IsReady == true` — both readiness loops iterate an empty list, so readiness is vacuously satisfied. Asserted as-is to pin the behaviour; raised as DEF-004 and later confirmed end to end through the UI (UI-GV-01/02)

UT-AS-12 OpenWorkspaceAsync delegates to the repository

PASS

COVERS

Delegation

PRE

Repo stubbed to return a known assessment id

STEPS

1. Call `OpenWorkspaceAsync(changeRequestId)`

EXPECTED

Returns the id; `GetOrCreateAsync` called once

ACTUAL

As expected

Unit — inherited suites

50 cases authored by the development team, executed and verified by QA. All passing.

Scoring — 11 cases ALL PASS

Guards the "controls mitigate, never eliminate" invariant in C#, as defence in depth on top of the database's own CHECK constraints.

ID	CASE	VERIFIES
UT-SC-01..03	UpsertConfig rejects out-of-range mitigation factor (-0.1, 1, 1.5)	US-10.1 AC2 — a factor ≥ 1.0 would let residual reach zero
UT-SC-04..06	UpsertConfig accepts in-range factor (0, 0.5, 0.99)	The boundary is < 1.0 , not ≤
UT-SC-07	Calculate throws if the repo returns a non-positive residual	The C# guard fires even if the DB somehow allowed it
UT-SC-08	Calculate returns the score when residual is positive	Happy path
UT-SC-09	Override rejects a non-positive residual	An analyst cannot override risk to zero
UT-SC-10	Override rejects a blank reason	US-7.2 — reason is mandatory
UT-SC-11	Override accepts a positive residual with a reason	Happy path

Committee — 14 cases ALL PASS

ID	CASE	VERIFIES
UT-CM-01	Vote rejected when not yet routed to committee	The finalize gate cannot be bypassed
UT-CM-02	No resolution below quorum	Epic 10 <code>CommitteeQuorum</code> rule
UT-CM-03..09	Conservative resolution rule (7 vote combinations)	Reject > Defer > ApproveWithConditions > Approved — one dissenting reject blocks approval
UT-CM-10	Conditions text merged from every approve-with-conditions vote	US-8.2 AC2 — nothing dropped
UT-CM-11	An already-decided assessment is not re-resolved	Idempotence
UT-CM-12	Go-live <code>true</code> pushed to Mock Systems on approval	Phase 3 feedback loop
UT-CM-13	Go-live <code>false</code> pushed on rejection	Feedback loop, negative case
UT-CM-14	Feedback push skipped when no entity was linked at intake	No-op rather than error

Note: a fifteenth behaviour — a Mock Systems outage must not fail a vote — is covered within the same file and passes; it is counted inside the 14 above.

Change requests — 7 cases ALL PASS

ID	CASE	VERIFIES
UT-CR-01	Submit triggers data ingestion with the linked Mock System ids	Snapshot is taken at intake
UT-CR-02..04	Attach accepts PDF, DOCX, XLSX	US-1.2 — supported formats
UT-CR-05	Attach rejects an unsupported content type	US-1.2 — clear rejection
UT-CR-06	File attach rejects unsupported type <i>without touching blob storage</i>	Validation precedes I/O
UT-CR-07	File attach uploads, extracts, then persists	Correct ordering of the three steps

Data ingestion — 5 cases ALL PASS

ID	CASE	VERIFIES
UT-DI-01	No-op when no entity was linked at intake	No spurious calls
UT-DI-02	Snapshot saved when the customer resolves	Happy path
UT-DI-03	Partial resolution still saves whatever resolved	A stale id does not lose the rest
UT-DI-04	A Mock Systems outage does not fail intake	Resilience — intake is more important than enrichment
UT-DI-05	GetSnapshot delegates to the repo	Delegation

AI clients — 11 cases ALL PASS

The citation-fabrication guards. These are the closest thing the repo has to a test of "the model cannot invent things".

ID	CASE	VERIFIES
UT-AI-01	Category mapping never calls the model when no categories are allowed	US-2.1 AC3 — flags rather than guesses; also zero cost
UT-AI-02	Drops a proposal outside the allowed list	Structural guard — the model cannot introduce a category
UT-AI-03	Parses a markdown-fenced response	Real models wrap JSON in fences
UT-AI-04	Degrades to empty on malformed JSON	No crash, no guess
UT-AI-05	Includes external context in the prompt when supplied	Mock Systems snapshot reaches the model
UT-AI-06	Extraction never calls the model for empty document text	Early return
UT-AI-07	Parses fields including the <code>needsReview</code> flag	US-4.1 — low confidence is surfaced
UT-AI-08	Extraction degrades to empty on malformed JSON	No silent guess
UT-AI-09	Narrative parses text and <code>unsupportedClaims</code> separately	US-5.1 AC3 — unsupported claims never folded into narrative
UT-AI-10	Narrative falls back to a reviewable result on an unparseable response	Fails toward human review
UT-AI-11	Regeneration feedback reaches the prompt	US-5.2

Document processing — 2 cases

ALL PASS

ID	CASE	VERIFIES
UT-DP-01	Constructor does not throw when unconfigured	A regression guard — config is validated at call time
UT-DP-02	Upload throws only at call time when unconfigured	The app still starts without blob config

API — smoke

3 cases · `tests/e2e/api/smoke.spec.js` · authored by QA. Pre-req: API on :5210 with a seeded database.

API-SM-01 Every response uses the OperationResult envelope

PASS

PRE

API reachable

STEPS

1. `GET /api/Ping`
2. Parse the JSON body

EXPECTED

200; body has `isSuccessful: true` and a `data` property

ACTUAL

As expected — the contract both the webapp and these tests rely on

API-SM-02 The seeded users cover all four roles

PASS

PRE

`seed_dev_users.sql` applied

STEPS

1. `GET /api/User`
2. Group by role

EXPECTED

≥ 6 users, with at least one each of ProductOwner, Analyst, CommitteeMember, Admin

ACTUAL

6 users, 4 roles — Priya Owens, Amara Chen, Sam Okafor, Jordan Blake, Riley Voss, Taylor Finch

API-SM-03 The FFIEC framework is seeded with its four categories

PASS

COVERS

CLAUDE.md — decomposition must use a real, citable framework

STEPS

1. `GET /api/CategoryMapping/Categories`
2. Extract the `code` of each

EXPECTED

Contains `PRODUCTS_SERVICES`, `CUSTOMERS_ENTITIES`, `GEOGRAPHIC_LOCATIONS`, `DELIVERY_CHANNELS`

ACTUAL

All four present — a partial seed would silently weaken every mapping downstream

API — authorization

9 cases · `tests/e2e/api/authorization.spec.js` · authored by QA. Six record live defects.

API-AU-01 The user directory is not readable without a credential

KNOWN DEFECT · DEF-001

PRE

API running; no Authorization header sent

STEPS

1. `GET /api/User` with no credential of any kind

EXPECTED

401 Unauthorized

ACTUAL

200 OK returning all six users with display names, email addresses and roles. `DevBypassAuthHandler` authenticates every request as `system-dev`, so the blanket `[Authorize]` on every controller proves nothing

API-AU-02 Ping stays deliberately open

PASS

STEPS

1. `GET /api/Ping`

EXPECTED

200 with `data: "pong"`

ACTUAL

As expected. Deliberate counter-example: `PingController` carries no `[Authorize]` *by design*, so "everything is open" and "this one is open on purpose" stay distinguishable

API-AU-03 A Product Owner cannot be recorded as a committee voter

KNOWN DEFECT · DEF-002

COVERS

US-8.2 AC5

PRE

Product Owner's user id resolved from `/api/User`

STEPS

1. `POST /api/Committee/Vote` with `committeeMemberUserId` = the Product Owner's id

EXPECTED

403 Forbidden — refused on authorization grounds before any lookup

ACTUAL

400 — rejected for workflow reasons; the role was never consulted

API-AU-04 A Product Owner cannot finalize an assessment

KNOWN DEFECT • DEF-002

COVERS

US-6.3

STEPS

1. POST /api/Assessment/{id}/Finalize with actorUserId = Product Owner

EXPECTED

403

ACTUAL

500

API-AU-05 A Product Owner cannot change scoring configuration

KNOWN DEFECT • DEF-002

COVERS

US-10.1

STEPS

1. POST /api/Scoring/Config with actorUserId = Product Owner

EXPECTED

403

ACTUAL

500

SCOPE OF API-AU-03/04/05 – STATE THIS WHEN PRESENTING

All three used a deliberately non-existent id, so each was rejected (or crashed) on *data* grounds before role could have mattered. They prove the role check does not run **first**. They do **not** prove a request against a valid, committee-routed assessment would be accepted — confirming that needs a seeded PendingCommittee assessment and remains open work. API-AU-01 carries no such caveat and is the stronger evidence.

API-AU-06 A scoring config that would zero out residual risk is rejected

PASS

COVERS

US-10.1 AC2

STEPS

1. POST /api/Scoring/Config with maxMitigationFactor: 1.0

EXPECTED

Not 200

ACTUAL

Rejected. Kept deliberately: proves the API *does* validate, so the finding above stays an authorization gap rather than "the API accepts anything"

API-AU-07 A malformed GUID is rejected by routing

PASS

STEPS

1. GET /api/Assessment/not-a-guid/Readiness

EXPECTED

404 — the route constraint rejects it before any handler runs

ACTUAL

As expected

API-AU-08 An unknown risk category is a client error, not a server error

KNOWN DEFECT • DEF-003

STEPS

1. POST /api/Scoring/Config with a well-formed but non-existent riskCategoryId

EXPECTED

Any 4xx

ACTUAL

500. Server log: Npgsql.PostgresException 23503 ... violates foreign key constraint scoring_config_risk_category_id_fkey . OperationResult already has NotFound / BadRequest shapes for exactly this

API-AU-09 Finalizing an unknown assessment is a client error

KNOWN DEFECT • DEF-003

STEPS

1. POST /api/Assessment/{unknown-guid}/Finalize

EXPECTED

404

ACTUAL

500

UI – role journeys

3 cases · `tests/e2e/ui/journeys.spec.js` · serial. Pre-req: full stack including the Vite dev server on :3000.

Sign-in needs no automation: while Auth0 is unconfigured, `DevUserContext.jsx` reads the acting user from `localStorage.devUserId`, so seeding that key before first paint *is* the sign-in. Each journey then clicks through the dev-auth gate.

UI-JN-01 A Product Owner raises a change request	PASS
COVERS US-1.1 AC4, US-1.3 PRE Stack running; acting user = Priya Owens (ProductOwner) STEPS 1. Open the app; dismiss "Continue without signing in" 2. Click nav Submit Request 3. Open the change-type combobox <i>inside the first form</i>; choose Product 4. Fill <code>#title</code> with a timestamped wire-transfer product title 5. Fill <code>#description</code> and <code>#details</code> (JSON) 6. Click Submit request 7. Click nav My Requests EXPECTED A confirmation matching <code>Submitted as CR-\d{4}-\d{5}</code>; the request appears in My Requests with type <i>Product</i> and status <i>Submitted</i> ACTUAL As expected — <code>CR-2026-00001</code> assigned and listed	

UI-JN-02 An FCRM Analyst assesses and finalizes it

PASS

COVERS

US-1.1, US-9.1, US-6.3

PRE

UI-JN-01 completed; acting user = Amara Chen (Analyst)

STEPS

1. Click nav **Assessments**; locate the request row
2. Click **Open**; *retry once* — see DEF-006
3. Assert all seven workspace tabs are present
4. Open the **Audit** tab
5. Open the **Finalize** tab; click **Finalize**
6. Click **Route to committee**

EXPECTED

The request reaches the analyst inbox; Categories / Policy / Extraction / Narrative / Scoring / Finalize / Audit all reachable; the audit trail already shows `ChangeRequest.Created by Priya Owens`; status becomes Finalized

ACTUAL

As expected. **Note:** step 2 needs a retry because of DEF-006 — asserted separately in UI-GV-03 rather than silently absorbed here

UI-JN-03 The Risk Committee reviews and votes

PASS

COVERS

US-8.1, US-8.2 AC5

PRE

UI-JN-02 completed; acting user = Jordan Blake (CommitteeMember)

STEPS

1. Click nav **Committee Queue**; locate the request row
2. Click **Review**
3. Select **Approve** in the vote combobox
4. Click **Submit vote**

EXPECTED

The finalized assessment is in the queue; the vote is recorded against the member *by name*; "No votes yet." disappears

ACTUAL

As expected — the individual voter is named, never aggregated anonymously

UI – governance defects

3 cases · `tests/e2e/ui/governance.spec.js` . All three record live defects.

UI-GV-01 The Finalize tab refuses an assessment with nothing reviewed	KNOWN DEFECT · DEF-004
COVERS CLAUDE.md – "don't let an AI output reach the committee stage without a review gate in code" PRE A fresh change request created via the API; acting user = Analyst STEPS 1. Open the assessment workspace for that request 2. Categories tab – confirm "No categories mapped yet." 3. Finalize tab – read the readiness panel EXPECTED "Ready to finalize" is not shown ACTUAL "Ready to finalize." is displayed with zero categories, zero policy reliance, zero extracted fields, zero narrative and zero scores	
UI-GV-02 An unreviewed assessment cannot be routed to the committee queue	KNOWN DEFECT · DEF-004
PRE A fresh change request; nothing assessed STEPS 1. As Analyst: open the workspace, go to Finalize 2. Click Finalize, then Route to committee 3. Switch to Jordan Blake (CommitteeMember) 4. Open Committee Queue and look for the request EXPECTED The request is absent from the queue ACTUAL Present, openable, with a live "Submit vote" button. The committee is asked for an accountable decision on a document containing nothing but a title. This is the most serious finding in the product.	

UI-GV-03 Two concurrent opens of the same request both succeed

KNOWN DEFECT · DEF-006

PRE

A fresh change request with no assessment row yet

STEPS

1. Fire two simultaneous `POST /api/Assessment/OpenWorkspace/{id}` via `Promise.all`
2. Collect both status codes

EXPECTED

`[200, 200]` — the function must be idempotent under concurrent callers

ACTUAL

One returns **500**. `func_getOrCreateAssessment` does SELECT-then-INSERT with no `ON CONFLICT`; both callers read `NULL`, both insert, the loser violates `UNIQUE (change_request_id)` → Npgsql `23505`. Because React StrictMode double-invokes effects, an analyst hits this on *every* first click

AI evaluation cases

12 cases · `evals/datasets/*.json` · inherited, executed by QA. 2 pass, 10 blocked.

The harness scores the three production AI clients against fixed datasets. Without a configured provider, the two deterministic cases still run; the rest report **Skip** rather than Fail — a missing key is not the same claim as "the model got it wrong".

ID	CASE	SCORING METHOD	STATUS
EV-CM-01	Wire transfer product → Products & Services	Set-overlap on FFIEC codes	BLOCKED
EV-CM-02	KYC vendor onboarding → Customers & Entities	Set-overlap	BLOCKED
EV-CM-03	Geographic expansion into a FATF-monitored jurisdiction	Set-overlap	BLOCKED
EV-CM-04	MSB customer segment → Customers & Entities	Set-overlap	BLOCKED
EV-CM-05	No framework mapping — empty allowed list must return empty	Deterministic, no model call	PASS
EV-DE-01	Vendor document, clear fields	Exact/substring match	BLOCKED
EV-DE-02	Geography document	Exact/substring match	BLOCKED
EV-DE-03	Ambiguous jurisdiction must set <code>needsReview</code>	Flag assertion	BLOCKED
EV-DE-04	Empty document — must not call the model	Deterministic, no model call	PASS
EV-NR-01	Well-grounded narrative cites what it was given	Citation-keyword proxy	BLOCKED
EV-NR-02	No grounding supplied — response well-formed	Well-formedness only	BLOCKED
EV-NR-03	Regeneration incorporates analyst feedback	Keyword presence	BLOCKED

WHAT NARRATIVE SCORING DOES NOT VERIFY

EV-NR-02 confirms the model returned a non-empty, parseable response — *nothing more*. It does not independently verify that the model avoided fabricating a claim. That property is a prompt-level instruction enforced by the client's contract and, ultimately, by the analyst review gate downstream. The eval harness's own README states this plainly, which is the right call: a narrative eval that quietly claimed more than it checks would be worse than no eval at all.

Observations

What running these 92 cases actually taught us, beyond the pass/fail counts.

OBS-1 · The unit suite predicted a defect the UI later confirmed

UT-AS-11 was written as a characterization test — pinning behaviour that looked wrong but belonged to another module's owner. Days later, UI-GV-01 and UI-GV-02 confirmed the same defect end to end in the browser, with a committee member able to vote on an empty assessment. **The two suites found the same thing from opposite directions**, which is the strongest form of corroboration available without a formal proof.

OBS-2 · Green means nothing until you have broken something on purpose

The mutation check is the single most useful artefact in this register. Deleting the finalization gate failed exactly five cases — not four, not fifteen. That precision is what distinguishes a suite that tests behaviour from one that merely exercises code.

OBS-3 · One test was deleted for being flaky, and that was the right call

A browser-level version of UI-GV-03 passed on one full-suite run and failed in isolation — the race only fires when the two StrictMode requests genuinely overlap. **A known-defect marker that oscillates is worse than no test**, because it trains the team to ignore the signal. It was replaced with a deterministic `Promise.all` version that also makes the stronger claim: the defect is in the stored function, not in React.

OBS-4 · Counter-evidence is what makes findings credible

API-AU-02, API-AU-06 and API-AU-07 all pass, and all sit deliberately inside the defect file. They prove the API *does* authenticate one route by design, *does* validate the mitigation factor, and *does* reject malformed input. Without them, DEF-002 reads as "the API is wide open" — an overstatement that would get argued down and take the true part with it.

OBS-5 · The largest coverage gap is structural, not an oversight

The `audit_event` append-only trigger and the `CHECK (residual_rating > 0)` constraint are the repo's flagship governance claims, and **not one of these 92 cases can reach them**. No HTTP endpoint attempts an `UPDATE audit_event`, so no API or UI test can ever exercise that trigger, and unit tests mock the database away entirely. This needs a Testcontainers suite, and it is the top priority.

OBS-6 · The role model exists and is honoured only in the UI

The frontend filters navigation by role — a Product Owner never sees the Committee Queue. The API enforces none of it. That combination is *worse* than having no role model, because the product looks access-controlled to anyone evaluating it from the screens.

OBS-7 · Two environment issues would break a fresh clone

- `ops/setup-local-db.sh` assumes `psql` on the host. On a Docker-only machine it fails partway; both schemas had to be applied through `docker exec` instead.
- The `control` table is created but never seeded (DEF-005), so `GET /api/Scoring/Controls` returns `[]` for every category and US-7.1 AC3's explainability requirement cannot be met.

Defect register

ID	FINDING	SEVERITY	CASES
DEF-001	User directory readable with no credential	HIGH	API-AU-01
DEF-002	No role enforcement; actor self-declared in the request body	HIGH	API-AU-03/04/05
DEF-003	Unknown ids surface as HTTP 500	MEDIUM	API-AU-08/09
DEF-004	Empty assessment finalizes and reaches the committee	HIGH	UT-AS-11, UI-GV-01/02
DEF-005	<code>control</code> table never seeded	MEDIUM	manual
DEF-006	<code>func_getOrCreateAssessment</code> race — first open always 500s	MEDIUM	UI-GV-03

Traceability

User story → the cases proving it. Gaps are listed as gaps.

USER STORY	CASES	STATUS
US-1.1 Submit a change request	UI-JN-01, UT-CR-01	COVERED
US-1.2 Attach supporting documents	UT-CR-02..07	COVERED
US-1.3 Track my request status	UI-JN-01	COVERED
US-2.1 Auto-map risk categories	UT-AI-01..05, EV-CM-01..05	PARTIAL – EVALS BLOCKED
US-2.2 Override proposed categories	UT-AS-09	PARTIAL – NO REASON-MANDATORY TEST
US-3.1 Surface relevant policy	—	GAP
US-3.2 Confirm or reject surfaced policy	UT-AS-03	COVERED
US-4.1 Extract structured data	UT-AI-06..08, EV-DE-01..04	PARTIAL – EVALS BLOCKED
US-4.2 Correct extracted data	—	GAP
US-5.1 Generate a draft assessment	UT-AI-09/10, UT-AS-01	COVERED
US-5.2 Regenerate with feedback	UT-AI-11, EV-NR-03	PARTIAL
US-6.1/6.2 Edit & override history	—	GAP – NEEDS DB TESTS
US-6.3 Finalize the assessment	UT-AS-01..08, UI-JN-02	COVERED
US-7.1 Risk score with breakdown	UT-SC-07/08	PARTIAL – DEF-005 BLOCKS EXPLAINABILITY
US-7.2 Override a calculated score	UT-SC-09..11	COVERED
US-8.1 Route to committee	UI-JN-02/03	COVERED
US-8.2 Cast a committee vote	UT-CM-01..10, UI-JN-03	COVERED
US-8.3 Record the final decision	UT-CM-03..13	COVERED
US-9.1 Reconstruct full history	UI-JN-02	PARTIAL – NO EXPORT TEST
US-9.2 Guarantee immutability	—	GAP – UNREACHABLE WITHOUT TESTCONTAINERS
US-10.1 Configure scoring parameters	UT-SC-01..06, API-AU-06	COVERED

USER STORY	CASES	STATUS
US-10.2 Configure workflow rules	—	GAP

THE HONEST SUMMARY

Of 22 user stories, **12 are covered, 6 partially, and 4 have no coverage at all**. The most consequential gap is US-9.2 — the immutable audit trail, this product's flagship governance claim — which no test in this register can reach. That is a structural limit of the current suites, not an oversight, and closing it is the top recommendation.

All results recorded from executions against a running local stack on 12 September 2026.
Suite stability confirmed over three consecutive full Playwright runs and four unit runs.
Companion documents: *QA & BA Strategy* and *QA Codebase Walkthrough*.